

# Staqtapp 1X

## Core API Quick Reference

Engine version 1.5.2 | Documentation release 1.5.3 | Programmer and AI-agent operational surface

|                                                                                                     |                                                                                                        |
|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Non-halting</b><br>Ordinary application failures return false-valued result objects.             | <b>Multiple VFS integration</b><br>Select or explicitly target independent storage files.              |
| <b>Transactional durability</b><br>Atomic streamed writes, revisions, integrity maps, and recovery. | <b>High-speed reads and writes</b><br>mmap spans, .sqti indexing, typed ranges, and optional batching. |

**Use this reference for:** configuration, file selection, string and typed values, transactions, optional batching, bounded reads, revisions, integrity, recovery, maintenance, multiprocessing, and diagnostics.

**Critical transaction scope:** [run\\_transaction\(\)](#) is single-VFS, mutation-only, all-or-nothing, and read-your-own-writes. It is not a general container for reads, maintenance, recovery, or multi-VFS work.

## 1. Configure and select storage

|                                                        |                                                                                                                                                                                                                                                          |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>configure(**kwargs)</b>                             | Configure process-level settings. Principal options include <code>storage_dir</code> , <code>json_backend</code> , <code>write_batching</code> , <code>batch_max_operations</code> , <code>batch_max_wait_ms</code> , and <code>batch_max_bytes</code> . |
| <b>makevfs(vfsFileName, directoryName, folderName)</b> | Create a VFS and select it as the active target.                                                                                                                                                                                                         |
| <b>setpath(vfsFileName, directoryName, folderName)</b> | Select an existing VFS.                                                                                                                                                                                                                                  |
| <b>open_vfs(name, directory, folder)</b>               | Advanced context manager for an explicit VFS. A failed <code>__enter__</code> cannot safely use the normal false-valued result pattern.                                                                                                                  |
| <b>json_backend_info()</b>                             | Report active typed-JSON backend, acceleration, fallback, and canonical-equivalence status.                                                                                                                                                              |

```
import staqtapp
```

```
staqtapp.configure(storage_dir="./data", json_backend="auto")
staqtapp.makevfs("accounts", "Production", "Primary")
```

## 2. Universal result handling

|                          |                                                                                                                                                                 |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CallFailure</b>       | Immutable, false-valued ordinary API failure with <code>api</code> , <code>error_type</code> , <code>message</code> , and <code>argument</code> representation. |
| <b>TransactionResult</b> | Transaction-wide result. False-valued when any operation rejects the complete transaction.                                                                      |
| <b>QueuedResult</b>      | Accepted into an optional process-local write batch; queued is not durable.                                                                                     |
| <b>BatchFlushResult</b>  | Result of crossing the batching durability boundary.                                                                                                            |

```
result = staqtapp.get_value("missing")
if not result:
    print(result.error_type, result.message)

staqtapp.set_value("next", "execution continues")
```

## 3. Legacy string variables

|                                     |                                       |
|-------------------------------------|---------------------------------------|
| <b>addvar(name, data)</b>           | Create one legacy string variable.    |
| <b>appvar(names, datas, locks)</b>  | Create multiple legacy variables.     |
| <b>changevar(name, newData)</b>     | Replace one variable.                 |
| <b>renamevar_stx(name, newName)</b> | Rename one variable.                  |
| <b>removevar(name)</b>              | Delete one variable.                  |
| <b>joinvars(newName, names)</b>     | Create a variable by joining sources. |
| <b>listvars()</b>                   | List variable names.                  |
| <b>listfiles()</b>                  | Return lightweight VFS metadata.      |
| <b>findvar(name)</b>                | Test variable existence.              |

|                                            |                                             |
|--------------------------------------------|---------------------------------------------|
| <b>findvar_stx(names, stalkName)</b>       | Retained multi-variable/stalk-aware lookup. |
| <b>loadvar(isAllNumbers, name, mode)</b>   | Load and decode a legacy variable.          |
| <b>vardata_stx(isRegex, names, search)</b> | Search payloads.                            |
| <b>stalkvar(name, data)</b>                | Record or update stalk/history data.        |
| <b>corevar(mode, name, booleanList)</b>    | Retained core-variable boolean codec.       |

## 4. Explicit typed values

|                                                  |                                                                            |
|--------------------------------------------------|----------------------------------------------------------------------------|
| <code>set_value(name, value)</code>              | Create or replace a deterministic typed value.                             |
| <code>get_value(name, expected_type=None)</code> | Decode a typed value; <code>expected_type</code> uses exact type identity. |
| <code>inspect_value(name)</code>                 | Return typed-envelope metadata.                                            |
| <code>validate_value(name)</code>                | Validate typed envelope and canonical encoding.                            |
| <code>encode_value(text)</code>                  | Legacy text encoding helper.                                               |

```
staqtapp.set_value("config", {
    "workers": 4,
    "impedance": 3.5 - 2j,
    "payload": b"\x00\xff",
})
config = staqtapp.get_value("config", dict)
```

## 5. Bounded and indexed reads

|                                                             |                                                                              |
|-------------------------------------------------------------|------------------------------------------------------------------------------|
| <code>read_payload_range(name, start=0, length=None)</code> | Read raw payload bytes directly from the mmap-backed span.                   |
| <code>read_range(name, start=0, length=None)</code>         | Read a canonical typed-value byte range with bounded Base64-window decoding. |
| <code>iter_value(name, chunk_size=1048576)</code>           | Iterate canonical typed bytes in chunks.                                     |
| <code>read_index_info()</code>                              | Inspect the validated optional .sqti sidecar.                                |
| <code>rebuild_read_index()</code>                           | Rebuild and atomically publish the .sqti index.                              |

```
staqtapp.rebuild_read_index()
chunk = staqtapp.read_payload_range("blob", 4096, 1024)
typed_slice = staqtapp.read_range("config", 0, 256)
```

## 6. Explicit transactions

|                                                                 |                                                                                                     |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>run_transaction(calls)</code>                             | One selected VFS; mutation-only; ordered; read-your-own-writes; all-or-nothing; one durable commit. |
| <code>run_vfs_transaction(vfs, directory, folder, calls)</code> | Same transaction model against an explicit VFS identity.                                            |

```
result = staqtapp.run_transaction([
    ("set_value", ("balance", 400)),
    ("set_value", ("status", "paid")),
    ("removevar", ("temporary",)),
])

if not result:
    print(result.failure.error_type)
```

## 7. Transaction scope and exclusions

|                            |                                                                               |
|----------------------------|-------------------------------------------------------------------------------|
| <b>Supported mutations</b> | addvar, appvar, changevar, renamevar_stx, removevar, joinvars, and set_value. |
|----------------------------|-------------------------------------------------------------------------------|

|                             |                                                                                                     |
|-----------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Not supported inside</b> | Reads, list operations, integrity, rollback, recovery, maintenance, diagnostics, or multi-VFS work. |
| <b>Failure semantics</b>    | One rejected operation means no transaction operation commits.                                      |
| <b>Performance rule</b>     | Use for two or more related mutations or many writes to the same VFS.                               |

## 8. Optional adaptive write batching

|                              |                                                                        |
|------------------------------|------------------------------------------------------------------------|
| <b>flush_writes()</b>        | Durability barrier for all process-local queued writes.                |
| <b>pending_writes()</b>      | Inspect queued/in-flight batch state.                                  |
| <b>write_batching_info()</b> | Return settings and batching counters.                                 |
| <b>QueuedResult</b>          | Accepted but not durable. Do not treat as a committed acknowledgement. |

```
staqtapp.configure(
    write_batching=True,
    batch_max_operations=100,
    batch_max_wait_ms=5,
)
receipt = staqtapp.set_value("event:1", {"type": "login"})
committed = staqtapp.flush_writes()
```

## 9. Locks and operation protection

|                                                 |                                                 |
|-------------------------------------------------|-------------------------------------------------|
| <b>lockvar(name, functionName)</b>              | Deny a named mutation operation for a variable. |
| <b>locklist(name)</b>                           | List operation locks.                           |
| <b>lockdel(isDeleteAll, name, functionName)</b> | Remove one or all operation locks.              |
| <b>keyvar(name, functionName)</b>               | Test whether an operation is locked.            |

## 10. Revisions and rollback

|                                    |                                                                                  |
|------------------------------------|----------------------------------------------------------------------------------|
| <b>list_revisions(limit=None)</b>  | List immutable content-addressed revision events.                                |
| <b>rollback_revision(revision)</b> | Restore exact bytes from a validated immutable revision and publish a new event. |
| <b>prune_revisions(keep=32)</b>    | Retain a bounded revision timeline.                                              |
| <b>revision_storage_report()</b>   | Report timeline, objects, referenced bytes, and reclaimable bytes.               |

```
revisions = staqtapp.list_revisions(limit=10)
result = staqtapp.rollback_revision(revisions[-1]["revision"])
```

## 11. Verification, integrity, and recovery

|                                                                        |                                                               |
|------------------------------------------------------------------------|---------------------------------------------------------------|
| <b>verify_vfs(...)</b>                                                 | Perform complete structural and SHA-256 VFS verification.     |
| <b>rebuild_integrity_map()</b>                                         | Build the advisory checksummed region map.                    |
| <b>verify_integrity(deep=True)</b>                                     | Validate map, VFS identity, boundaries, and region checksums. |
| <b>integrity_report(rebuild_if_missing=False, deep=True)</b>           | Return integrity status and localized failures.               |
| <b>repair_vfs(strategy="surgical", source="latest-valid-revision")</b> | Restore only checksum-proven damaged variable regions.        |
| <b>recover_vfs(...)</b>                                                | Recover from the local .bak image after validation.           |
| <b>migrate_vfs(...)</b>                                                | Run the non-destructive migration path.                       |

```
report = staqtapp.verify_integrity(deep=True)
if not report["ok"]:
    repaired = staqtapp.repair_vfs()
```

## 12. Maintenance

|                                       |                                                                                                |
|---------------------------------------|------------------------------------------------------------------------------------------------|
| <b>optimize_vfs()</b>                 | Canonicalize when needed, preserve logical values, and refresh acceleration metadata.          |
| <b>compact_vfs(keep_revisions=32)</b> | Optimize, safely prune revision history, and remove recognized disposable temporary artifacts. |

```
staqtapp.optimize_vfs()
staqtapp.compact_vfs(keep_revisions=32)
```

## 13. Multiprocessing and dynamic invocation

|                                                                                    |                                                                        |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>invoke_api(api_name, args=(), kwargs=None)</b>                                  | Invoke one allow-listed public API by name.                            |
| <b>map_api_calls(calls, processes=None, start_method="spawn", chunksize=1)</b>     | Map independent calls. Successful operations may commit independently. |
| <b>invoke_vfs_api(vfs, directory, folder, api_name, args=(), kwargs=None)</b>      | Invoke one API against an explicit VFS.                                |
| <b>map_vfs_api_calls(calls, processes=None, start_method="spawn", chunksize=1)</b> | Map explicit-VFS calls through spawn-safe workers.                     |

```
result = staqtapp.invoke_api("get_value", args=("config",))
results = staqtapp.map_api_calls([
    ("findvar", ("alpha",)),
    ("findvar", ("beta",)),
])
```



## 14. Diagnostics

|                                        |                                                                    |
|----------------------------------------|--------------------------------------------------------------------|
| <code>recent_events(limit=100)</code>  | Return bounded recent diagnostic events.                           |
| <code>diagnostic_counts()</code>       | Return diagnostic event counts.                                    |
| <code>listfiles()</code>               | Lightweight file-generation metadata; not a full integrity digest. |
| <code>json_backend_info()</code>       | Inspect typed JSON routing.                                        |
| <code>write_batching_info()</code>     | Inspect batching state and counters.                               |
| <code>read_index_info()</code>         | Inspect .sqti index status.                                        |
| <code>revision_storage_report()</code> | Inspect revision storage growth.                                   |

## 15. AI-agent operating pattern

Use explicit VFS targeting when an agent may operate on several files. Prefer transactions for related mutations, flush queued writes before reporting durable success, and verify integrity before attempting recovery.

```
result = staqtapp.run_vfs_transaction(
    "accounts", "Production", "Primary",
    [
        ("set_value", ("balance", 500)),
        ("set_value", ("status", "active")),
    ],
)

if not result:
    return {"ok": False, "error": result.failure.message}

report = staqtapp.verify_integrity(deep=True)
return {"ok": bool(report["ok"]), "durable": True}
```

Never describe a QueuedResult as durable. Never use `map_api_calls()` when all-or-nothing behavior is required. Never place recovery or maintenance calls inside `run_transaction()`.

## 16. Compact decision guide

|                              |                                                                                                 |
|------------------------------|-------------------------------------------------------------------------------------------------|
| One direct durable write     | Use the ordinary mutation call with write batching disabled.                                    |
| Several related writes       | Use <code>run_transaction()</code> .                                                            |
| High-rate independent writes | Enable optional batching and cross <code>flush_writes()</code> before durable acknowledgement.  |
| Read part of a large payload | Use <code>read_payload_range()</code> or <code>read_range()</code> .                            |
| Fast repeated lookups        | Use the validated <code>.sqti</code> index; <code>rebuild_read_index()</code> if needed.        |
| Suspected corruption         | <code>verify_integrity()</code> then <code>repair_vfs()</code> only when provenance is trusted. |
| Storage growth               | <code>revision_storage_report()</code> , then <code>compact_vfs()</code> .                      |
| Independent parallel calls   | <code>map_api_calls()</code> or <code>map_vfs_api_calls()</code> ; not an atomic batch.         |

## Disabled compatibility names

Several historical names remain import-compatible but intentionally unsupported: `sqttpp_rd1`, `lambdalist`, `lambdavar`, `registry`, `genicvar`, `darkvar`, `revar`, `addtree_stx`, `addbranch_stx`, `getbranch_stx`, and `pojishon`. New code should not use them.